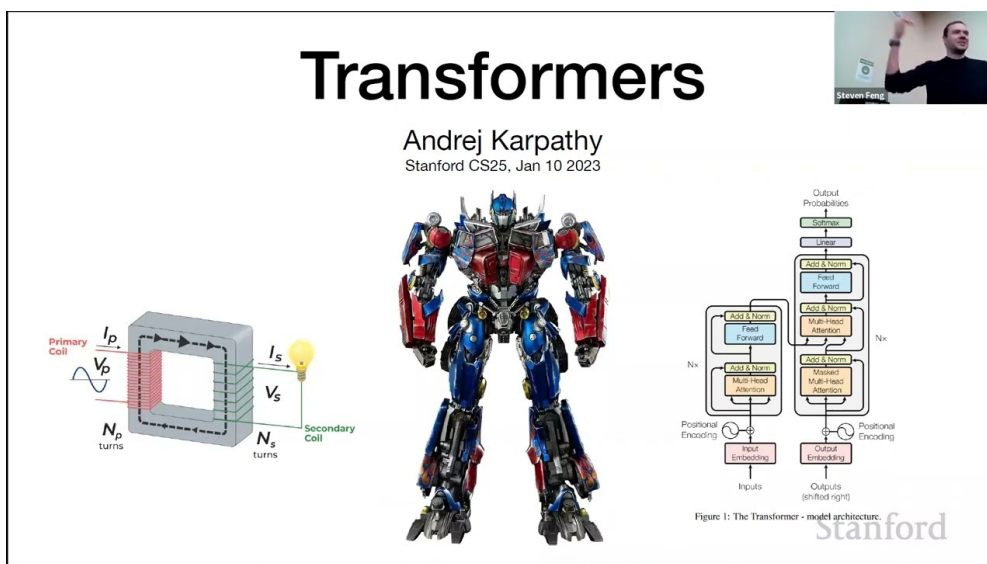


课程笔记

[CS25] Introduction to Transformers — Andrej Karpathy

基于公开课程资料整理

2023



视频作者/频道: Stanford CS25

发布日期: 2023

视频时长: 约 60 分钟

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：课程概述	2
2 Transformer 的核心架构	2
2.1 编码器-解码器结构	2
2.2 自注意力机制详解	2
2.3 Transformer Block 的结构	2
2.4 本章小结	3
3 位置编码	3
3.1 为什么需要位置编码?	3
3.2 位置编码的类型	3
3.3 本章小结	3
4 训练与应用	3
4.1 预训练范式	3
4.2 ChatGPT 与 GPT 系列	4
4.3 本章小结	4
5 总结与延伸	4
5.1 拓展阅读	4

1 引言：课程概述

Andrej Karpathy 以 CS25 “Transformers United” 的开场讲座形式，系统地介绍了 Transformer 架构的核心构建模块。Karpathy 彼时正从 Tesla AI 团队转入 OpenAI，同时维护着 minGPT/-nanoGPT 等教育项目。

2 Transformer 的核心架构

2.1 编码器-解码器结构

原始 Transformer (“Attention Is All You Need”) 采用编码器-解码器结构：

编码器与解码器的区别

- **编码器**：双向注意力——每个 token 可以看到序列中的所有其他 token。用于理解/表征任务 (BERT)
- **解码器**：因果注意力——每个 token 只能看到自己和之前的 token (三角形掩码)。用于生成任务 (GPT)
- **交叉注意力**：解码器的 token 可以注意编码器的所有输出 (不受因果限制)

三大 Transformer 变体

- **仅编码器**：BERT——双向理解，适合分类、NLU
- **仅解码器**：GPT 系列——自回归生成，当前最主流
- **编码器-解码器**：T5、BART——翻译、摘要等 seq2seq 任务

后续发展表明，**仅解码器**架构凭借其简洁性和强大的 scaling 特性成为主导。

2.2 自注意力机制详解

自注意力的消息传递视角

Transformer 可以看作一个**消息传递网络**：

- 每个 token 是图中的一个节点
- 编码器中是全连接图 (所有节点互相连接)
- 解码器中是有向无环图 (只连接到之前的节点)
- 交叉注意力连接解码器节点到所有编码器节点

自注意力就是在这个图上进行一轮消息传递——每个节点从其邻居收集信息并更新自己。

2.3 Transformer Block 的结构

每个 Transformer block 包含：

1. **Layer Normalization** (通常使用 Pre-Norm 变体)
2. **多头自注意力**
3. **残差连接**
4. **Layer Normalization**
5. **MLP** (前馈网络, 通常是 2 层, 中间维度 $4\times$)
6. **残差连接**

Transformer 的稳定性

Karpathy 指出, Transformer 的架构在提出 5 年多后几乎没有改变——尽管无数研究者尝试改进。这种架构稳定性在深度学习中是罕见的, 说明原始设计已经找到了某种“最优点”。

2.4 本章小结

Transformer 通过自注意力、残差连接和 MLP 的简洁组合, 构建了一个强大的序列处理架构。

3 位置编码

3.1 为什么需要位置编码?

自注意力操作本身是**置换不变的**——打乱 token 顺序不影响输出。但序列顺序在语言中至关重要 (“狗咬人” $\neq$ “人咬狗”)。位置编码将位置信息注入模型。

3.2 位置编码的类型

- **绝对正弦位置编码**: 原始 Transformer 使用
- **可学习位置编码**: GPT 系列使用
- **旋转位置编码 (RoPE)**: 编码相对位置, 已成为主流
- **ALiBi**: 通过注意力分数中的线性偏置编码位置

3.3 本章小结

位置编码是 Transformer 中看似简单但设计选择丰富的组件, 直接影响模型处理长序列的能力。

4 训练与应用

4.1 预训练范式

- **语言模型预训练**: 在大规模文本上做下一个 token 预测
- **指令微调**: 在指令-回答对上微调
- **RLHF**: 通过人类反馈的强化学习对齐模型行为

4.2 ChatGPT 与 GPT 系列

Karpathy 提到了 ChatGPT 的出现及其影响：

- 本质上是 GPT-3.5 + 指令微调 + RLHF
- 在产品层面的打磨（对话界面）是其成功的关键
- minGPT → nanoGPT 的演进：让 GPT 训练更加民主化

4.3 本章小结

从预训练到 RLHF 的训练流水线已经成为构建大语言模型的标准范式。

5 总结与延伸

Karpathy 的讲座为 CS25 系列奠定了基础：Transformer 的核心组件（自注意力、位置编码、MLP、残差连接）构成了一个简洁但极其强大的架构。后续讲座将展示这一架构如何在各个领域产生变革性影响。

5.1 拓展阅读

- Vaswani et al., “Attention Is All You Need,” NeurIPS 2017
- Radford et al., “Language Models are Unsupervised Multitask Learners” (GPT-2), 2019
- Karpathy, “nanoGPT,” GitHub repository
- Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” NeurIPS 2022